

Ejercicio 1: Crítica a los patrones de diseño.

Luego de realizar una investigación respecto a las contras que poseen los patrones de diseño, una de las críticas más recurrentes es que muchos patrones estructurales y de comportamiento solo existen para compensar las limitaciones de los lenguajes orientados a objetos menos flexibles tal como Java o C++.

Un ejemplo concreto que podemos encontrar en el patrón de diseño de Strategy es que se usa para intercambiar algoritmos encapsulándolos en clases. En lenguajes modernos que soportan funciones de primera clase como Python o JavaScript. Este patrón a menudo es redundante, ya que simplemente puedes pasar una función o un *lambda* como argumento en lugar de crear toda una jerarquía de clases.

Otra crítica mencionada en los patrones de diseño es el aumento de complejidad al querer aplicar estos patrones de diseño en todas partes, incluso cuando existe una solución simple.

Esto produce capas de abstracción innecesarias que hacen que el código sea más difícil de leer y mantener, un ejemplo de donde podemos encontrar esto es al tratar de implementar el patrón Abstract Factory para instanciar objetos en un script pequeño o en una aplicación que nunca va a requerir múltiples familias de objetos. Lo que podría haberse resuelto con una simple función condicional termina requiriendo múltiples clases y superclases.

Algunos patrones priorizan la flexibilidad y la abstracción por encima de la eficiencia. Ensamblar objetos mediante múltiples capas puede consumir más memoria y tiempo de procesamiento. Un ejemplo común se encuentra en el patrón Decorator envuelve objetos dentro de otros. Si se abusa de él, se puede terminar con un objeto envuelto en muchas capas, lo que dificulta la depuración del código y añade un ligero costo de rendimiento al llamar a los métodos.